2025 IEEE International Conference on Advanced Robotics (ICAR)
2-5 December 2025. San Juan, Argentina

2025 IEEE International Conference on Advanced Robotics (ICAR) | 979-8-3315-7809-1/25/$31.00 ©2025 IEEE | DOI: 10.1109/ICAR65334.2025.11338658

VIGS-Fusion: Fast Gaussian Splatting SLAM processed onboard a small quadrirotor

Abdoullah Ndoye[1,2], Amaury Nègre[1], Nicolas Marchand[1] and Franck Ruffier[2]

***Abstract*— Real-time dense visual-inertial SLAM remains a major challenge for resource-constrained UAVs, especially when using 3D Gaussian Splatting (3DGS). While 3DGS enables high-quality 3D reconstruction, it suffers from high computational cost and weak tracking robustness. In this paper, we introduce VIGS-Fusion, an efficient 3DGS SLAM system that can operate fully onboard. Our method computes a robust and accurate pose from a single tightly-coupled optimization problem that incorporates RGB-D data, preintegrated IMU measurements, velocity, and IMU bias. By using a second-order approximation and a loss based on the observed image gradients for pose estimation, we significantly reduced computational time while maintaining high tracking performance. We also provide a real-world dataset collected with our compact quadrotor, capturing RGB-D and IMU measurements. Experimental results demonstrated accurate and robust tracking while achieving over a 30x reduction in computation time for the tracking compared to state-of-the-art 3DGS SLAM systems. For the first time, we demonstrated real-time 3D Gaussian Splatting SLAM running entirely onboard a 6-inch UAV. The code is available at: https://github.com/AbdoullahNdoye/VIGS-Fusion.git.**

I. INTRODUCTION

Autonomous flight of Unmanned Aerial Vehicles (UAVs) requires accurate localization. In applications such as exploration of unknown environments, survey missions, and infrastructure inspection, it also requires building a map of the surrounding environment. Recent advances in 3D scene representation techniques, such as 3D Gaussian Splatting (3DGS) [1], allow high-quality map construction, making them attractive for Simultaneous Localization and Mapping (SLAM). Traditional map representations like surfels [2], [3], or voxels [4] can only reconstruct parts of a scene that have been seen by the camera. In contrast, 3DGS allows novel view synthesis, which can be beneficial for many applications in virtual reality (VR) and augmented reality (AR). One example application is VR-based teleoperation of UAVs in unfamiliar environments, such as mining sites [5], [6], where 3DGS SLAM can allow users to localize, build a photorealistic map of the scene, explore, and navigate the environment to perform tasks more effectively. 3DGS SLAM systems can be categorized into those using only monocular or RGB-D cameras and those that integrate additional sensors. Recent works [7], [8], [9] demonstrated effective visual SLAM using 3DGS with accurate tracking and map reconstruction with monocular or RGB-D cameras. In these works, the camera position was tracked by using simple gradient-based optimization, minimizing color and depth reconstruction errors. This process requires a large number of iterations, making these systems challenging to use for real-time applications. Additionally, the accuracy and robustness of these methods are highly dependent on the depth quality. The integration of data from LiDAR [10], [11], IMU [12], or both [13], [14] has also been studied, enhancing the performance of 3DGS systems. While LiDARs provide accurate depth measurements, their size and power consumption are often prohibitive for small UAVs. IMUs, in contrast, are lightweight but subject to motor vibrations, bias, and noise that must be considered.

Fig. 1. Our 6-inch quadrotor performing simultaneous localization and mapping using 3D Gaussian splatting onboard.

Despite these advances, most 3DGS-based systems are still limited in their applicability to small UAVs, where robust, accurate tracking, as well as real-time processing are crucial.

In this work, we propose VIGS-Fusion (Visual-Inertial Gaussian Splatting Fusion): a novel visual-inertial 3DGS SLAM system capable of real-time operation onboard a small UAV (here, a 6-inch propeller-driven quadrotor). Our method can perform robust and accurate pose tracking in 9 iterations, thus considerably reducing the computation time for resource-constrained platforms. We can summarize the main contributions of our paper as follows:

- We introduce a second-order optimization framework for 3DGS SLAM that directly minimizes photometric and geometric error using the observed image derivatives by resolving a non-linear least squares problem,

*Financial support was provided via the DarkNav project grant (ANR-20-CE33-0009) to N.M. and F.R. from ANR. The facilities for the experimental tests has been mainly provided by ROBOTEX 2.0 (Grants ROBOTEX ANR-10-EQPX-44-01 and TIRREX ANR-21-ESRE-0015).

[1]Abdoullah Ndoye, Amaury Nègre and Nicolas Marchand are with Univ. Grenoble Alpes, CNRS, GIPSA-lab, Grenoble, France {abdoullah.ndoye, amaury.negre, nicolas.marchand} @grenoble-inp.fr

[2]Franck Ruffier was with Aix-Marseille Université, CNRS, ISM, Marseille, France and is now with ENSTA | IP Paris, CNRS, Lab-STICC, 29200, Brest, France franck.ruffier@cnrs.fr

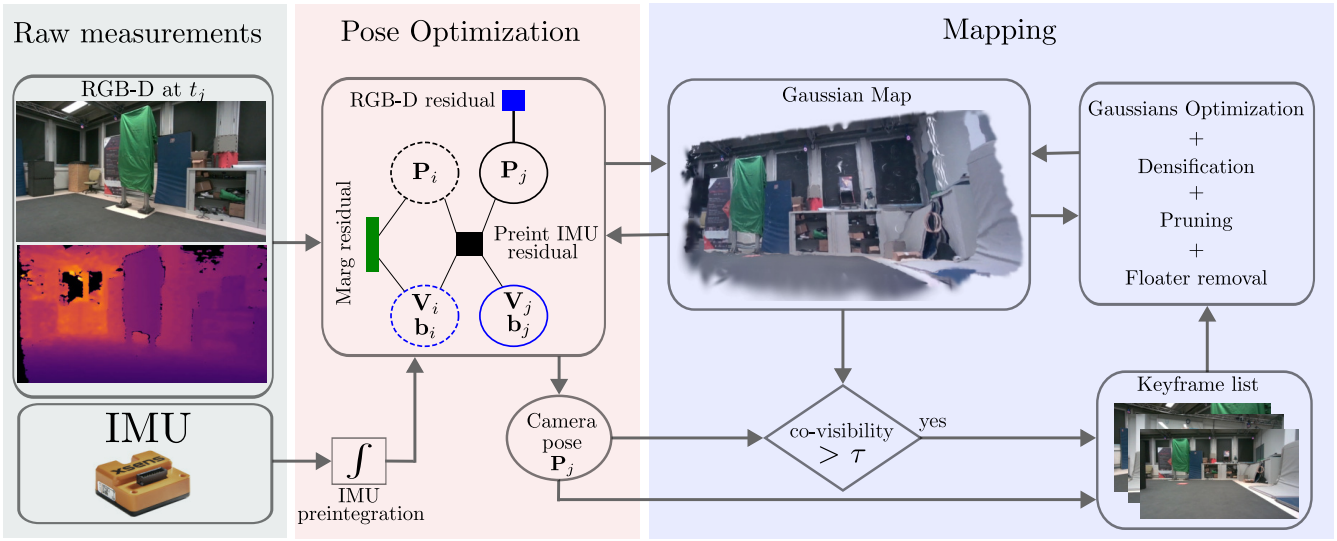

Fig. 2. VIGS-Fusion SLAM overview. The UAV's state (pose, velocity, and IMU biases) is estimated by tightly coupling visual and inertial measurements through a single joint optimization problem including RGB-D, IMU, and marginalization residuals. New keyframes are added when their co-visibility with the current frame exceeds a threshold τ. At each new keyframe, new Gaussians are added to the map (densification), and degenerated Gaussians are removed (pruning). Gaussians created with inaccurate depth may result in floaters, which are detected and eliminated (floater removal). The Gaussian parameters are optimized using simple gradient descent.

reducing tracking time by up to 30× compared to state of the art 3DGS SLAM algorithms.

- Inspired by [15], the pose is estimated by tightly coupling IMU and RGB-D measurements. We solve a single optimization problem estimating the camera pose, velocity, and IMU bias allowing robust pose tracking on our UAV dataset.
- We release a dataset comprising onboard RGB-D and IMU measurements with ground truth for evaluating 3DGS SLAM systems, collected using our 6-inch quadrotor.

To validate the effectiveness of VIGS-Fusion, we compare it with two state-of-the-art 3DGS SLAM systems: *MM3DGS* [12] and *SplaTAM* [8]. We show that our method achieves robust and precise localization, with only 9 iterations, resulting in 30× reduction in tracking time, and better localization. Real-time performance is demonstrated with experiments onboard our UAV equipped with a Jetson Orin NX.

II. Related Work

Recent advances in 3DGS [1] have attracted significant attention in the SLAM community, leading to the development of various 3DGS-based SLAM systems.

3DGS SLAM systems using RGB or RGB-D cameras. MonoGS [7] introduced the first 3DGS SLAM system using a single monocular camera. While it achieved accurate results in small, room-scale environments, its tracking performance deteriorated in larger, real-world scenes. With RGB-D cameras, 3DGS SLAM systems [8], [16], [17], [18], [19], [20] showed improved tracking performance (see review [21]). SplaTAM [8] introduced a simplified Gaussian representation resulting in a more efficient SLAM and precise camera tracking. GS-SLAM [9] used a coarse optimization step to choose only reliable Gaussians for better pose estimation. These 3DGS SLAM methods computed the camera pose by minimizing the error between the real (observed) and the rendered images using first-order gradient descent. As a result, they required a large number of tracking iterations, which considerably increase computation time. Gaussian-SLAM [16] organized 3D Gaussians into sub-maps that are locally optimized and used a frame-to-model tracking algorithm. GS-ICP [17] combined ICP [22] with Gaussian Splatting to improve the speed and robustness of tracking. However, these systems, which rely on depth information, are very sensitive to depth noise, limiting their performance in real-world scenarios. Other works such as CaRtGS [23] and Photo-SLAM [24] adopted hybrid approaches, combining 3DGS with feature-based methods such as ORB-SLAM3 [25]. These methods maintained separate maps for Gaussians and features, leading to higher memory usage and computational costs.

3DGS SLAM systems using additional sensors. Relying solely on an RGB-D camera is not sufficient to achieve good 3DGS SLAM performance due to motion blur, inaccurate depth estimation, or low-texture scenes, among other challenges. Fusing data from other sensors is necessary to increase tracking performance. [10], [11], [13], [14], [26] have used LiDAR. However, the weight and power consumption of LiDAR systems make these approaches unsuitable for small UAV applications. In *MM3DGS* [12], the authors proposed a method that uses preintegrated inertial data to initialize the camera pose at each frame and assist with pose optimization (using gradient descent). However, in *MM3DGS*, camera pose estimation requires one hundred iterations to converge. Additionally, the inertial data were not tightly coupled with the visual data, and the IMU biases were not estimated,

reducing robustness when using lower-quality IMUs.

In contrast, our approach tightly fuses inertial data and visual measurements in a joint optimization framework. The pose is estimated by minimizing RGB-D, IMU, and marginalization residuals within a Levenberg-Marquardt solver, significantly reducing the number of iterations compared to first-order gradient descent while improving robustness and precision. RGB-D, IMU, and marginalization residuals are discussed in the next section.

III. VIGS-Fusion SLAM Method

Fig. 2 shows an overview of our VIGS-Fusion method. From RGB-D and IMU measurements, we optimize the UAV pose and update the Gaussian map using selected keyframes. The pose optimization is represented using a factor graph formulation (see pose optimization block in Fig. 2).

A. 3D Gaussian Splatting

3DGS uses a collection of Gaussian primitives with adjustable parameters to represent a scene [1]. Each 3D Gaussian primitive i is characterized by its mean $\boldsymbol{\mu}_i$, covariance matrix $\boldsymbol{\Sigma}_i$, opacity o_i, and color $\mathbf{c}_i$. For rendering, 3D Gaussians are projected into the 2D image space using the Jacobian of the linear approximation of the projective transformation $\mathbf{J}_t$ and the rotation from world to camera frame $\mathbf{W}$. The projected covariance matrix $\boldsymbol{\Sigma}'_i$, and the Gaussian projected mean $\boldsymbol{\mu}'_i$ are given by:

$$\boldsymbol{\Sigma}'_i = \mathbf{J}_t \mathbf{W} \boldsymbol{\Sigma}_i \mathbf{W}^\top \mathbf{J}_t^\top \quad , \quad \boldsymbol{\mu}'_i = \pi(\mathbf{T}_{cw} \cdot \boldsymbol{\mu}_i), \quad (1)$$

π denotes the projection operation, $\boldsymbol{\mu}_i$ is the position of a Gaussian i in the world frame and $\mathbf{T}_{cw}$ is the transformation from world to camera.

The color of a pixel $\mathbf{x}$ is obtained by α-blending $\mathcal{N}$ ordered points overlapping the pixel [1]:

$$C(\mathbf{x}) = \sum_{i \in \mathcal{N}} \mathbf{c}_i \alpha_i \prod_{j=1}^{i-1} (1 - \alpha_j), \quad (2)$$

α_i is the density of the projected Gaussian at pixel $\mathbf{x}$:

$$\alpha_i(\mathbf{x}) = o_i . e^{-\frac{1}{2}(\mathbf{x}-\boldsymbol{\mu}'_i)^\top \boldsymbol{\Sigma}'^{-1}_i (\mathbf{x}-\boldsymbol{\mu}'_i)}, \quad (3)$$

In our system, the depth of a Gaussian primitive is rasterized as done in [27]:

$$d_i(\mathbf{x}) = z_i + \hat{\mathbf{p}}_\mathbf{i}(\boldsymbol{\mu}'_i - \mathbf{x}), \quad (4)$$

where z_i is the depth of the Gaussian center and $\hat{\mathbf{p}}_\mathbf{i}$ is a vector that takes into account the variation of the depth across different pixels covered by the same Gaussian. More details can be found in [27]. In order to preserve discontinuities, the rendered depth $d(\mathbf{x})$ for a given pixel is computed as the median of the depths of all overlapping Gaussians.

B. UAV Pose Optimization

Fig. 2 shows the UAV pose optimization in a factor graph formulation. The system state at time k is defined as:

$$\mathbf{x}_\mathrm{k} = [\mathbf{P}_k, \mathbf{V}_\mathrm{k}, \mathbf{b}_a^k, \mathbf{b}_g^k] \quad (5)$$

where $\mathbf{P}_k \in SE(3)$ represents the UAV pose, comprising translation $\mathbf{p}_k \in \mathbb{R}^3$ and rotation $\mathbf{R}_k \in SO(3)$; $\mathbf{V}_k$ is the velocity; and $\mathbf{b}_a^k$, $\mathbf{b}_g^k$ are the accelerometer and gyroscope biases, respectively. Our method jointly optimizes pose, velocity, and IMU biases by tightly coupling IMU and RGB-D measurements in a single optimization problem. Estimating biases online improves robustness, especially when using low-quality IMUs, addressing a key limitation of existing visual-inertial 3DGS SLAM systems.

The overall cost function $C(\mathbf{x}_k)$ combines RGB-D residual ($r_{\text{RGB-D}}$), IMU residual (r_{IMU}), and a marginalization residual (r_{Marg}), which retains information from previous states:

$$C(\mathbf{x}_k) = r_{\text{RGB-D}} + r_{\text{IMU}} + r_{\text{Marg}} \quad (6)$$

We minimize this cost using the Levenberg–Marquardt algorithm. To the best of our knowledge, our approach is the first to apply a second-order approximation to the RGB-D residual for 3DGS SLAM, enabling faster convergence. Specifically, our approach achieves more than 30× reduction in computation time compared to existing 3DGS SLAM algorithms at their maximum performance, which typically relied on first-order gradient descent.

1) Levenberg–Marquardt algorithm: We briefly recall the formulation of a non-linear least squares problems [28]:

$$\min_{\mathbf{x}} F(\mathbf{x}) = \frac{1}{2} \| r(\mathbf{x}) \|_2^2, \quad (7)$$

$F(\mathbf{x})$ is the objective function, $\mathbf{x} \in \mathbb{R}^n$ the state variable, and $r(\mathbf{x}) : \mathbb{R}^n \mapsto \mathbb{R}$ is the residual, typically a nonlinear function.

The Levenberg–Marquardt algorithm is used to solve Eq. (7) iteratively. For each iteration k: (i) we compute the Jacobian $J(\mathbf{x}_k) = \frac{\partial r(\mathbf{x}_k)}{\partial \mathbf{x}_k}$ and (ii) solve the equation:

$$(\mathbf{H} + \lambda I) \Delta \mathbf{x}_k = -\mathbf{J}(\mathbf{x}_k)^\top r(\mathbf{x}_k) \quad (8)$$

where $\mathbf{H} = \mathbf{J}(\mathbf{x}_k)\mathbf{J}(\mathbf{x}_k)^\top$ is the approximation of the Hessian matrix and λ the damping factor. (iii) The state is updated: $\mathbf{x}_{k+1} = \mathbf{x}_k + \Delta \mathbf{x}_k$. When $\Delta \mathbf{x}_k$ is small, the algorithm is stopped.

In VIGS-Fusion, we used Lie algebra to compute the Jacobian with respect to the pose. Each pose $\mathbf{P}_k \in SE(3)$ in the manifold is parameterized in the tangent space using its corresponding Lie algebra representation $\xi_k = [\rho, w]^\top \in \mathfrak{se}(3)$, where $\rho \in \mathbb{R}^3$ is the translation and $w \in \mathbb{R}^3$ represents the rotation. During optimization, the pose is updated using the exponential map:

$$\mathbf{P}_{k+1} = \exp(\delta \xi_k) \mathbf{P}_k$$

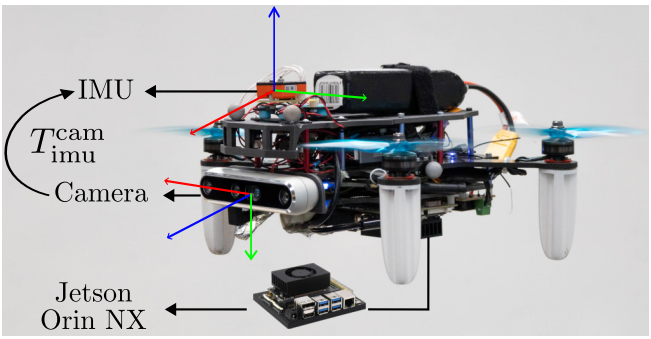

Fig. 3. Our 6-inch propeller-driven quadrotor used for dataset collection and onboard realtime SLAM, is equipped with Intel® RealSense™ D455 RGB-D camera, MTi-610 IMU, and a PX4 flight controller. An NVIDIA® Jetson™ Orin™ NX performs all processing on board. The $SE(3)$ Transformation matrix between the camera and the IMU is provided in the dataset.

2) RGB-D residual: The RGB-D residual is composed of photometric and geometric residuals.

$$r_{\text{RGB-D}} = r_{photo} + r_{\text{geo}} \tag{9}$$

$r_{\text{photo}} = RGB(\mathbf{x}) - C(\mathbf{x})$, where $C(\mathbf{x})$ is the rendered color and $RGB(\mathbf{x})$ the observed color. Similarly, $r_{\text{geo}} = D(\mathbf{x}) - d(\mathbf{x})$, with $d(\mathbf{x})$ the rendered depth at pixel $\mathbf{x}$ (computed as the median of the Gaussian depths) and $D(\mathbf{x})$ the measured depth. The Jacobians of these residuals were computed on the manifold with respect to the camera pose ξ_c.

Existing 3DGS SLAM algorithms use simple gradient descent and compute the Jacobian using the model ($C(\mathbf{x})$ and $D(\mathbf{x})$). This involves differentiating the color composition which is a sum over weighted Gaussians on relevant pixels resulting in more parameters being used, increased computation time and memory usage. Our method is faster using only the observed image derivatives that are easily computed in the 2D image space. Therefore, for the Jacobians, we differentiated only the observed color $RGB(\mathbf{x})$ and the measured depth $D(\mathbf{x})$.

Jacobian of photometric error r_{photo}:

$$\mathbf{J}_{\xi_c}^{r_{\text{photo}}} = \frac{\mathcal{D} r_{\text{photo}}}{\mathcal{D}\xi_c} = \frac{\mathcal{D}(RGB(\mathbf{x}) - C(\mathbf{x}))}{\mathcal{D}\xi_c} = \frac{\mathcal{D}RGB(\mathbf{x})}{\mathcal{D}\xi_c} \tag{10}$$

We use only the observed image gradients for computing this Jacobian. By applying the chain rule:

$$\frac{\mathcal{D}RGB(\mathbf{x})}{\mathcal{D}\xi_c} = \frac{\mathcal{D}RGB(\mathbf{x})}{\mathcal{D}\mathbf{u}} \frac{\mathcal{D}\mathbf{u}}{\mathcal{D}\mathbf{u}_c} \frac{\mathcal{D}\mathbf{u}_c}{\mathcal{D}\xi_c}, \tag{11}$$

where $\mathbf{u} = (u_x, u_y)$ is the 2D pixel location, $\mathbf{u}_c$ the corresponding 3D point in the camera frame. $\frac{\mathcal{D}\mathbf{u}}{\mathcal{D}\mathbf{u}_c} = \mathbf{J}_c$ is the Jacobian of the camera projection. $\frac{\mathcal{D}\mathbf{u}_c}{\mathcal{D}\xi_c} = [I, -\mathbf{u}_c^{\times}]$ (see [7]). $\frac{\mathcal{D}RGB(\mathbf{x})}{\mathcal{D}\mathbf{u}}$ is the gradient of the image:

$$\frac{\mathcal{D}RGB(\mathbf{x})}{\mathcal{D}\mathbf{u}} = \begin{bmatrix} \mathrm{g}_{\mathrm{u_x}} \\ \mathrm{g}_{\mathrm{u_y}} \end{bmatrix} = \begin{bmatrix} \mathrm{g}_{u_x}^R & \mathrm{g}_{u_x}^G & \mathrm{g}_{u_x}^B \\ \mathrm{g}_{u_y}^R & \mathrm{g}_{u_y}^G & \mathrm{g}_{u_y}^B \end{bmatrix} \tag{12}$$

where $\mathrm{g}_{u_x}^R$, $\mathrm{g}_{u_x}^G$, and $\mathrm{g}_{u_x}^B$ represent the horizontal gradients of the red, green, and blue channels, and $\mathrm{g}_{u_y}^R$, $\mathrm{g}_{u_y}^G$, and $\mathrm{g}_{u_y}^B$ their corresponding vertical gradients. The matrix $\frac{\mathcal{D}RGB(\mathbf{x})}{\mathcal{D}\mathbf{u}}$ thus captures how pixel color values change with respect to image-space displacements.

Jacobian of the geometric error r_{geo}:

$$\mathbf{J}_{\xi_c}^{r_{\text{geo}}} = \frac{\mathcal{D} r_{\text{geo}}}{\mathcal{D}\xi_c} = \frac{\mathcal{D}(D(\mathbf{x}) - d(\mathbf{x}))}{\mathcal{D}\xi_c} = \frac{\mathcal{D}D(\mathbf{x})}{\mathcal{D}\xi_c} \tag{13}$$

Applying the chain rule:

$$\frac{\mathcal{D}D(\mathbf{x})}{\mathcal{D}\xi_c} = \frac{\mathcal{D}D(\mathbf{x})}{\mathcal{D}\mathbf{u}_c} \frac{\mathcal{D}\mathbf{u}_c}{\mathcal{D}\xi_c} = \mathbf{r}[I - \mathbf{u}_c^{\times}] \tag{14}$$

where $\mathbf{r}$ is the ray direction:

$$\mathbf{r} = \begin{bmatrix} \frac{u_x - c_x}{f_x} & \frac{u_y - c_y}{f_y} 1 \end{bmatrix}^{\top} \tag{15}$$

f_x, f_y, c_x and c_y are the intrinsic camera parameters. Finally, for the RGB-D contribution, we can write:

$$\mathbf{J}_{\xi_c}^{r_{\text{RGB-D}}\top} \mathbf{J}_{\xi_c}^{r_{\text{RGB-D}}} = \mathbf{J}_{\xi_c}^{r_{\text{photo}}\top} \mathbf{J}_{\xi_c}^{r_{\text{photo}}} + \mathbf{J}_{\xi_c}^{r_{\text{geo}}\top} \mathbf{J}_{\xi_c}^{r_{\text{geo}}} \tag{16}$$

$$\mathbf{J}_{\xi_c}^{r_{\text{RGB-D}}\top} r_{\text{RGB-D}} = \mathbf{J}_{\xi_c}^{r_{\text{photo}}\top} r_{\text{photo}} + \mathbf{J}_{\xi_c}^{r_{\text{geo}}\top} r_{\text{geo}} \tag{17}$$

While minimizing the RGB-D residual provides a good basis for pose estimation, visual information alone can be affected by motion blur, inaccurate depth measurements, or low-texture environments, leading to reduced accuracy. To improve robustness and precision, we tightly integrated inertial measurements from an IMU.

3) IMU residual: The state of the UAV between two frames is predicted by preintegrating the IMU measurements into a single one over the corresponding time interval [29], [30]. These preintegrated measurements were used to initialize the visual system. The IMU residual r_{IMU} measures the difference between the predicted state, and actual state of the UAV estimated from the visual information:

$$r_{\text{IMU}} = \hat{\mathbf{x}}_k \boxminus \mathbf{x}_k \tag{18}$$

where $\hat{\mathbf{x}}_k$ is the predicted state from preintegration. $\boxminus$ is the boxminus operator. It computes the difference between two states on a manifold. The expression for the Jacobian of the IMU error with respect to the IMU pose ξ_i ($\mathbf{J}_{\xi_i}^{r_{\text{IMU}}}$) can be found in [31].

4) Marginalization residual: At time k, when a new frame arrives, a new state $\mathbf{x}_{\text{k}}$ is estimated. A naive approach would be to simply discard the old state $\mathbf{x}_{\text{k-1}}$. But if the state $\mathbf{x}_{\text{k-1}}$ is eliminated, we lose all past information about that state, and the estimation will converge by minimizing only the current error, ignoring past errors. We must add a prior to the previous state to retain information about past states. This is called marginalization. Our system integrates a marginalization residual r_{Marg} following the Schur complement-based marginalization method described in [31].

C. Visual Inertial Fusion

Recall the state of the system at a time k for VIGS-Fusion:

$$\mathbf{x}_{\text{k}} = [\mathbf{P}_k, \mathbf{V}_{\text{k}}, \mathbf{b}_a^k, \mathbf{b}_g^k] \tag{19}$$

The optimization is done in the IMU frame. In practice when fusing RGB-D and IMU, the transformation between the

Fig. 4. Qualitative results showing reconstructed 3D scene from keyframes renderings and two novels views generated from unseen camera perspectives. Generating novel views is important for 3DGS SLAM and benefits applications such as robot teleoperation in unfamiliar or partially mapped environments

camera and the IMU $T_{\text{imu}}^{\text{cam}} \in SE(3)$ (see Fig. 3) must be considered. The camera pose can be written:

$$\mathbf{p}^{\text{cam}} = \mathbf{p}^{\text{imu}} + \mathbf{R}^{\text{imu}}\mathbf{p}_{\text{imu}}^{\text{cam}} \tag{20}$$

$$\mathbf{R}^{\text{cam}} = \mathbf{R}^{\text{imu}}\mathbf{R}_{\text{imu}}^{\text{cam}} \tag{21}$$

where $\mathbf{R}^{\text{imu}}$ is the orientation of the IMU, $\mathbf{p}_{\text{imu}}^{\text{cam}}$ is the position of the camera in the IMU frame, and $\mathbf{R}_{\text{imu}}^{\text{cam}}$ the orientation of the camera in the IMU frame.

The RGB-D residual is computed with respect to the camera pose ξ_c. We apply chain rule to express this residual with respect to the IMU pose ξ_i:

$$\frac{\mathcal{D}r_{\text{RGB-D}}}{\mathcal{D}\xi_i} = \frac{\mathcal{D}r_{\text{RGB-D}}}{\mathcal{D}\xi_c} \cdot \frac{\mathcal{D}\xi_c}{\mathcal{D}\xi_i} = \mathbf{J}_{\xi_c}^{r_{\text{RGB-D}}} \cdot \mathbf{J}_{\xi_i}^{\xi_c} \tag{22}$$

This ensures that the RGB-D measurement updates are properly propagated to the IMU frame during optimization. The Jacobian $\mathbf{J}_{\xi_i}^{\xi_c}$ maps the derivatives from the camera frame to the IMU frame, and is given by:

$$\mathbf{J}_{\xi_i}^{\xi_c} = \begin{bmatrix} \mathbf{I}_{3\times3} & -\mathbf{R}^{\text{imu}}[\mathbf{p}_{\text{imu}}^{\text{cam}}]_\times \\ \mathbf{0}_{3\times3} & \mathbf{R}_{\text{imu}}^{\text{cam}\top} \end{bmatrix}, \tag{23}$$

We minimize the overall cost function (Eq. (6)) using the Levenberg–Marquardt algorithm by iteratively solving Eq. (8) on the manifold with:

$$\mathbf{H} = \mathbf{J}^\top\mathbf{J} = (\mathbf{J}_{\xi_c}^{r_{\text{RGB-D}}}\mathbf{J}_{\xi_i}^{\xi_c})^\top(\mathbf{J}_{\xi_c}^{r_{\text{RGB-D}}}\mathbf{J}_{\xi_i}^{\xi_c}) + \mathbf{J}_{\xi_i}^{r_{\text{IMU}}\top}\mathbf{J}_{\xi_i}^{r_{\text{IMU}}} + \mathbf{J}_{\xi_i}^{r_{\text{Marg}}\top}\mathbf{J}_{\xi_i}^{r_{\text{Marg}}} \tag{24}$$

$$\mathbf{J}^\top r = (\mathbf{J}_{\xi_c}^{r_{\text{RGB-D}}}\mathbf{J}_{\xi_i}^{\xi_c})^\top r_{\text{RGB-D}} + \mathbf{J}_{\xi_i}^{r_{\text{IMU}}\top} r_{\text{IMU}} + \mathbf{J}_{\xi_i}^{r_{\text{Marg}}\top} r_{\text{Marg}} \tag{25}$$

The UAV pose $\mathbf{P}_k$ is updated at each new iteration k until convergence:

$$\mathbf{P}_{k+1} = \exp(\delta\xi_k)\mathbf{P}_k \tag{26}$$

D. Mapping

For the mapping, the 3D Gaussians were initialized from one pixel or a set of pixels, using depth information to determine their initial position and size. The Gaussian map was continuously optimized and densified at each new keyframe. The keyframes were selected based on their co-visibility ratio with the current frame. Degenerated Gaussians (low opacity or small size) were eliminated (pruning). Using gradient descent, the parameters of the Gaussian primitives were optimized to achieve high-quality rendering.

During initialization and densification, due to the noise in the depth map, some Gaussians can be initialized in wrong positions often resulting in floaters. To address this issue, a floater removal step was applied. For each Gaussian i, we computed an outlier probability considering the estimated Gaussian depths $d_i(x)$ and the input camera depth values $D(x)$:

$$p_{floater}(i) = \frac{\sum_x \alpha_i(x) \cdot f(d_i(x), D(x))}{\sum_x \alpha_i(x)}$$

where

$$f(d, D) = \begin{cases} 1, & \text{if } d < 0.8 \cdot D \\ 0, & \text{otherwise} \end{cases}$$

The Gaussian was removed if $p_{floater}$ is greater than a given threshold (0.6 for our experimentations).

E. Implementation

Our algorithm was implemented using C++ and CUDA. For the optimization of the Gaussian map, we implemented the parallelization scheme done in [32] by processing 2D splats within tiles instead of pixels. This allows to gain a significant amount of time for the rendering. For the tracking, a coarse-to-fine approach was applied. The input images (color and depth) were stored into a pyramid with 3 levels, where each level corresponds to a progressively lower resolution of the image. During the pose optimization, we started from the coarsest to the finest level. We also kept only the contributions of pixels that had an opacity above a certain threshold. Ceres Solver [33] was used with the computed IMU, RGB-D, and Marginalization Jacobians. Although Ceres uses the CPU for calculations, the RGB-D Jacobian and residuals were calculated and aggregated on the GPU, so that Ceres works only with Jacobians of size 6×6.

IV. Experimental Results

A. UAV Platform

The compact UAV is a 6-inch propeller-driven quadrotor, equipped with an Intel® RealSense™ D455 depth camera and an Xsens MTi-610 IMU, with a total mass of 1.69 kg (see Fig. 3). Flight control is managed by a Durandal Flight Controller running PX4, while visual-inertial SLAM is processed on an embedded NVIDIA® Jetson™ Orin™

TABLE I

PERFORMANCE EVALUATION OF ONBOARD AND OFFLINE VIGS-FUSION VERSUS OFFLINE *MM3DGS* AND OFFLINE *SplaTAM* FOR DIFFERENT SEQUENCES OF OUR DATASET.

3DGS SLAM Method		Square				Circular				XYZ-Rot-obstacles				Cross-Loopy			
		ATE [cm]	PSNR [dB]	Track/ Img [ms]	Map/ Img [ms]	ATE [cm]	PSNR [dB]	Track/ Img [ms]	Map/ Img [ms]	ATE [cm]	PSNR [dB]	Track/ Img [ms]	Map/ Img [ms]	ATE [cm]	PSNR [dB]	Track/ Img [ms]	Map/ Img [ms]
MM3DGS offline*	9 iter	*Failed to converge*				*Failed to converge*				*Failed to converge*				*Failed to converge*			
	60 iter									105	19.94	790	1958				
	100 iter	2.99	26.55	860	1402	12.88	25.36	1048	1697	40.8	21.46	1279	2000	36.6	22.02	932	1561
SplaTAM offline*	9 iter	6.06	18.21	85	299	19.17	21.84	86	314	17.34	22.59	95	346	11.72	23.16	86	302
	60 iter	3.41	21.39	546	295	15.29	23.50	559	309	9.73	23.17	623	345	9.35	24.98	545	295
	100 iter	3.05	19.98	911	295	15.23	23.61	926	306	9.98	23.34	1024	341	9.346	25.18	895	289
VIGS-Fusion offline*	9 iter	2.42	25.74	16.29	112	6.59	24.00	17.47	97	8.54	24.04	14	97	3.72	25.78	17.72	110
VIGS-Fusion onboard**	9 iter	4.06	25.13	39.23	31	7.6	22.75	38.83	30.19	14.00	21.89	38.54	32.41	3.86	25.04	41.76	28.76

* The offline comparison was evaluated using an NVIDIA RTX 4090 graphics card.
** VIGS-Fusion ran here onboard Jetson ORIN NX. The resolution was reduced from 848×480px to 424×240px; the map was optimized using ADAM; the Gaussian initialization used a set of 3x3 pixels rather than a single pixel.

NX 16GB. The IMU sensor is mounted on a silent-block to absorb vibrations caused by the rotors. Additionally, the acceleration and gyroscopic data are processed through a digital biquad low-pass filter.

TABLE II

COMPARISON WITH SOME VISUAL-INERTIAL SLAM DATASETS

	Onboard the UAV	RGB	Depth	IMU
VIGS-Fusion (ours)	✓	✓	✓	✓
Euroc - MAV [34]	✓	✓	x	✓
TUM RGB-D[35]	x	✓	✓	✓
Bad slam [36]	x	✓	✓	✓
MM3DGS [12]	x	✓	✓	✓

B. Dataset

Many real-world datasets have been proposed for RGB-D SLAM. While TUM RGB-D [35] is one of the most used, it contains only accelerometer data, but no gyroscope measurements. [12] provides RGB-D and IMU measurements in their dataset, but it is derived from a wheeled robot platform with dynamics very different from those of a UAV. [34] proposed a dataset captured with a UAV that includes IMU data but provides only stereo data, not RGB-D. Table II summarizes the differences between these datasets.

Here, we propose a new dataset comprising eleven different sequences of RGB-D and full IMU data (accelerometer and gyroscope). The dataset is available in ROS bag format and enables a comprehensive evaluation of RGB-D-inertial SLAM for UAV-like platforms. It can be accessed online here: [37].

C. Baselines and Evaluation Metrics.

We compared our method (VIGS-Fusion) with two state-of-the-art 3DGS SLAM systems: *MM3DGS* [12], the first 3DGS system using RGB-D and IMU and SplaTAM [8] that uses only RGB-D. For offline comparison, we ran all the algorithms on an NVIDIA RTX 4090. We assessed performance using the following metrics: Absolute Trajectory Error (ATE) RMSE (in cm) for localization precision, Peak Signal-to-Noise Ratio (PSNR, in dB) for rendering quality, and total tracking and mapping times per image (Track/Img and Map/Img, in ms) for computational efficiency.

D. *Offline* Comparison with MM3DGS and SplaTAM

Tab. I presents a comparative evaluation of our method, VIGS-Fusion, against two state-of-the-art 3DGS SLAM algorithms: *MM3DGS* (RGB-D + IMU) and *SplaTAM*(RGB-D only), across four sequences from our UAV dataset. All experiments were conducted with an NVIDIA RTX 4090 GPU with RGB-D images at resolution 848×480. For the Map, gradient descent was used with 100 iterations for all three algorithms.

VIGS-Fusion, outperformed *MM3DGS* and *SplaTAM* in terms of ATE RMSE across all four sequences. In particular, for more complex scenarios involving translational and rotational motions near obstacles, such as the *XYZ-Rot-obstacles* sequence, VIGS-Fusion achieved an ATE of 8.54 cm, compared to 40.8 cm for *MM3DGS* and 9.98 cm for *SplaTAM* (using 100 tracking iterations for both baselines). Similarly, in the *Cross-Loopy* sequence, with loopy motion, VIGS-Fusion achieved an ATE of 3.72 cm compared to 36.6cm for *MM3DGS* and 9.34cm for *SplaTAM* (also with 100 tracking iterations). The major improvement in tracking accuracy is mainly due to the tight coupling of IMU and RGB-D measurements. By jointly optimizing pose, velocity and IMU biases within a single least square problem, VIGS-Fusion have robust pose estimates, even with aggressive UAV dynamics. Effective IMU bias estimation prevented drift, enhancing tracking robustness.

While *MM3DGS* achieved slightly higher PSNR in sim-

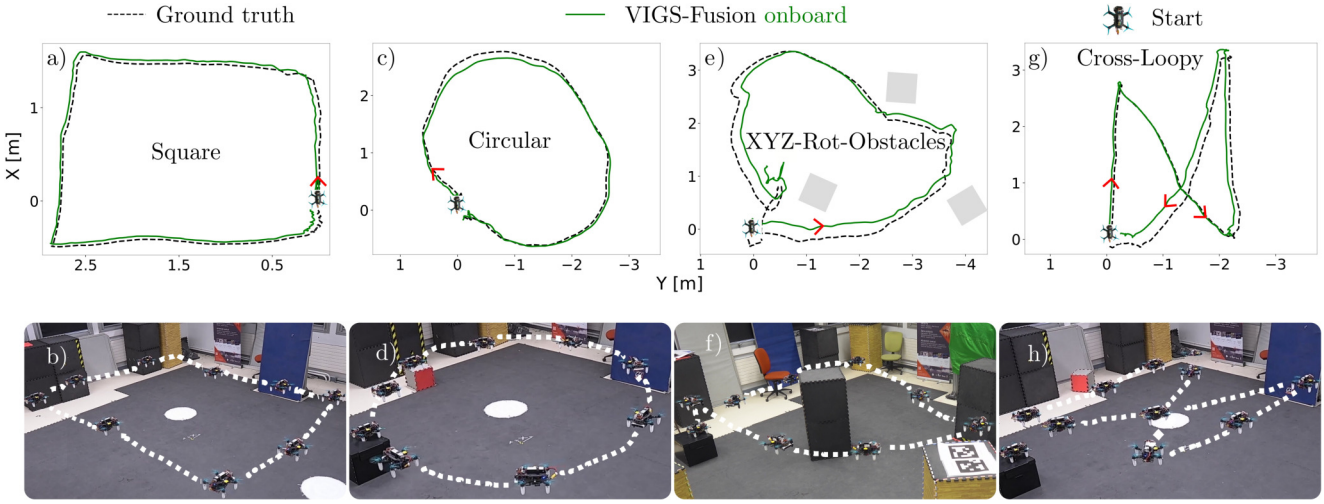

Fig. 5. Trajectory comparison between the ground truth and VIGS-Fusion onboard across four sequences from our dataset.

pler sequence like *Square* and *Circular*, VIGS-Fusion outperformed both baselines in complex sequences *XYZ-Rot-obstacles* and *Cross-Loopy*.

In term of computational time, VIGS-Fusion is especially faster. It converged after 9 iterations, achieving tracking times of 14–17 ms per image. This represents a 50× to 90× reduction in computation time compared to *MM3DGS* that mostly fail when the number of iteration is low (9 and 60 iteration across all sequences except *XYZ-Rot-obstacles*). Compared to *SplaTAM*, VIGS-Fusion had more than 30× reduction in computation time with 60 and 100 iterations and a 5× reduction with 9 iterations. But, with 9 iterations, *SplaTAM* experienced a big degradation in performances (increased ATE and reduced PSNR). The big reduction in the computation time stems from (i) our implemented approximation of the second order optimizer, (ii) the use of the image gradients instead of 3D Gaussians derivatives for the pose optimization. This allows also us to perform real time operation onboard our resource-constrained UAV.

E. VIGS-Fusion Onboard a Small UAV

VIGS-Fusion is capable of real-time operation onboard our 6-inch propeller-driven quadrotor, shown in Fig. 3. It is equipped with a Jetson Orin NX, which performed all the computations. To achieve real-time performance, we down-sampled images from 848×480 to 424×240. This resulted in reduced computation time with only minimal impact on performance (see Table I). We used two different threads: (i) a localization thread responsible for UAV pose tracking and (ii) a mapping thread for Gaussian optimization, densification, pruning, and floater removal. For the map optimization, we used the Adam optimizer [38].

Fig. 5 shows trajectory results of VIGS-Fusion onboard compared with the ground truth. Compared to offline results on NVIDIA RTX 4090, there is a slight reduction in performance, but it still performed better than *MM3DGS* and *SplaTAM* on *Circular* and *Cross-Loopy* sequences in terms of tracking precision. VIGS-Fusion achieved a frame rate of 14 FPS, enabling real-time 3DGS SLAM. Fig. 6 shows the difference in rendering quality between the full resolution (offline) and the down-sampled version used for real-time operations. Fig. 4 shows novel views generated from unseen camera perspectives during an experiment.

V. CONCLUSIONS

We propose VIGS-Fusion, a novel visual-inertial SLAM system leveraging 3D Gaussian Splatting (3DGS) for tracking and scene representation. VIGS-Fusion achieves accurate and robust tracking while significantly reducing tracking computation by up to 30× compared to state-of-the-art visual-inertial 3DGS SLAM systems under their optimal performance. This is enabled by (i) employing a second-order optimizer instead of first-order gradient descent, (ii) using image gradients that operate directly on raw pixel intensities for camera pose estimation, and (iii) tightly coupling of the IMU with the camera pose. VIGS-Fusion demonstrates real-time capability onboard a 6-inch propeller-driven quadrotor, making it suitable for applications such as VR-based teleoperation, where timely and accurate scene understanding is essential for performing remote tasks. In future work, we aim to improve rendering performance and explore loop-closure techniques to improve global map.

ACKNOWLEDGMENT

We thank Jonathan Dumon, Florian Pouthier, Pierre Susbielle, Alexis Offermann, Sylvain Durand, Ian Page and José-de-Jesus Castillo Zamora for their kind help and for the fruitful discussions.

REFERENCES

[1] B. Kerbl, G. Kopanas, T. Leimkühler, and G. Drettakis, "3d gaussian splatting for real-time radiance field rendering," *ACM Transactions on Graphics*, vol. 42, no. 4, July 2023. [Online]. Available: https://repo-sam.inria.fr/fungraph/3d-gaussian-splatting/

[2] H. Pfister, M. Zwicker, J. Van Baar, and M. Gross, "Surfels: Surface elements as rendering primitives," in *Proceedings of the 27th annual conference on Computer graphics and interactive techniques*, 2000, pp. 335–342.

[3] T. Whelan, S. Leutenegger, R. F. Salas-Moreno, B. Glocker, and A. J. Davison, "Elasticfusion: Dense slam without a pose graph." in *Robotics: science and systems*, vol. 11, no. 3. Rome, 2015.

Fig. 6. Rendering comparison between VIGS-Fusion offline and VIGS-Fusion onboard. While VIGS-Fusion onboard had a slight reduction in performance (Table. I), it achieved real-time operation onboard a small UAV and outperformed *MM3DGS* and *SplaTAM* in ATE on *Circular* and *Cross-Loopy* sequences.

[4] R. A. Newcombe, S. Izadi, O. Hilliges, D. Molyneaux, D. Kim, A. J. Davison, P. Kohi, J. Shotton, S. Hodges, and A. Fitzgibbon, "Kinectfusion: Real-time dense surface mapping and tracking," in *2011 10th IEEE international symposium on mixed and augmented reality*. Ieee, 2011, pp. 127–136.

[5] V. Patil and M. Hutter, "Radiance fields for robotic teleoperation," in *2024 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. IEEE, 2024, pp. 13 861–13 868.

[6] I. Page, P. Susbielle, O. Aycard, and P.-B. Wieber, "Real-time photorealistic mapping for situational awareness in robot teleoperation," 2025. [Online]. Available: https://arxiv.org/abs/2509.06433

[7] H. Matsuki, R. Murai, P. H. Kelly, and A. J. Davison, "Gaussian splatting slam," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, June 2024, pp. 18 039–18 048.

[8] N. Keetha, J. Karhade, K. M. Jatavallabhula, G. Yang, S. Scherer, D. Ramanan, and J. Luiten, "Splatam: Splat track & map 3d gaussians for dense rgb-d slam," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, June 2024, pp. 21 357–21 366.

[9] C. Yan, D. Qu, D. Xu, B. Zhao, Z. Wang, D. Wang, and X. Li, "Gs-slam: Dense visual slam with 3d gaussian splatting," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2024, pp. 19 595–19 604.

[10] C. Wu, Y. Duan, X. Zhang, Y. Sheng, J. Ji, and Y. Zhang, "Mm-gaussian: 3d gaussian-based multi-modal fusion for localization and reconstruction in unbounded scenes," *arXiv preprint arXiv:2404.04026*, 2024.

[11] R. Xiao, W. Liu, Y. Chen, and L. Hu, "Liv-gs: Lidar-vision integration for 3d gaussian splatting slam in outdoor environments," *IEEE Robotics and Automation Letters*, 2024.

[12] L. C. Sun, N. P. Bhatt, J. C. Liu, Z. Fan, Z. Wang, T. E. Humphreys, and U. Topcu, "Mm3dgs slam: Multi-modal 3d gaussian splatting for slam using vision, depth, and inertial measurements," *arXiv preprint arXiv:2404.00923*, 2024.

[13] S. Hong, J. He, X. Zheng, C. Zheng, and S. Shen, "Liv-gaussmap: Lidar-inertial-visual fusion for real-time 3d radiance field map rendering," *IEEE Robotics and Automation Letters*, 2024.

[14] X. Lang, L. Li, H. Zhang, F. Xiong, M. Xu, Y. Liu, X. Zuo, and J. Lv, "Gaussian-lic: Photo-realistic lidar-inertial-camera slam with 3d gaussian splatting," *arXiv preprint arXiv:2404.06926*, 2024.

[15] T. Laidlow, M. Bloesch, W. Li, and S. Leutenegger, "Dense rgb-d-inertial slam with map deformations," in *2017 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. IEEE, 2017, pp. 6741–6748.

[16] V. Yugay, Y. Li, T. Gevers, and M. R. Oswald, "Gaussian-slam: Photo-realistic dense slam with gaussian splatting," *arXiv preprint arXiv:2312.10070*, 2023.

[17] S. Ha, J. Yeon, and H. Yu, "Rgbd gs-icp slam," in *European Conference on Computer Vision*. Springer, 2024, pp. 180–197.

[18] S. Sun, M. Mielle, A. J. Lilienthal, and M. Magnusson, "High-fidelity slam using gaussian splatting with rendering-guided densification and regularized optimization," in *2024 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. IEEE, 2024, pp. 10 476–10 482.

[19] J. Hu, X. Chen, B. Feng, G. Li, L. Yang, H. Bao, G. Zhang, and Z. Cui, "Cg-slam: Efficient dense rgb-d slam in a consistent uncertainty-aware 3d gaussian field," in *European Conference on Computer Vision*. Springer, 2024, pp. 93–112.

[20] T. Deng, Y. Chen, L. Zhang, J. Yang, S. Yuan, J. Liu, D. Wang, H. Wang, and W. Chen, "Compact 3d gaussian splatting for dense visual slam," *arXiv preprint arXiv:2403.11247*, 2024.

[21] S. Zhu, G. Wang, D. Kong, and H. Wang, "3d gaussian splatting in robotics: A survey," *arXiv preprint arXiv:2410.12262*, 2024.

[22] A. Segal, D. Haehnel, and S. Thrun, "Generalized-icp." in *Robotics: science and systems*, vol. 2, no. 4. Seattle, WA, 2009, p. 435.

[23] D. Feng, Z. Chen, Y. Yin, S. Zhong, Y. Qi, and H. Chen, "Cartgs: Computational alignment for real-time gaussian splatting slam," 2024. [Online]. Available: https://arxiv.org/abs/2410.00486

[24] H. Huang, L. Li, H. Cheng, and S.-K. Yeung, "Photo-slam: Real-time simultaneous localization and photorealistic mapping for monocular stereo and rgb-d cameras," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2024, pp. 21 584–21 593.

[25] C. Campos, R. Elvira, J. J. G. Rodríguez, J. M. Montiel, and J. D. Tardós, "Orb-slam3: An accurate open-source library for visual, visual–inertial, and multimap slam," *IEEE transactions on robotics*, vol. 37, no. 6, pp. 1874–1890, 2021.

[26] P. Jiang, G. Pandey, and S. Saripalli, "3dgs-reloc: 3d gaussian splatting for map representation and visual relocalization," *arXiv preprint arXiv:2403.11367*, 2024.

[27] B. Zhang, C. Fang, R. Shrestha, Y. Liang, X. Long, and P. Tan, "Rade-gs: Rasterizing depth in gaussian splatting," *arXiv preprint arXiv:2406.01467*, 2024.

[28] X. Gao and T. Zhang, *Introduction to visual SLAM: from theory to practice*. Springer Nature, 2021.

[29] T. Qin, P. Li, and S. Shen, "Vins-mono: A robust and versatile monocular visual-inertial state estimator," *IEEE transactions on robotics*, vol. 34, no. 4, pp. 1004–1020, 2018.

[30] C. Forster, L. Carlone, F. Dellaert, and D. Scaramuzza, "Imu preintegration on manifold for efficient visual-inertial maximum-a-posteriori estimation," 2015.

[31] S. Leutenegger, S. Lynen, M. Bosse, R. Siegwart, and P. Furgale, "Keyframe-based visual–inertial odometry using nonlinear optimization," *The International Journal of Robotics Research*, vol. 34, no. 3, pp. 314–334, 2015.

[32] S. S. Mallick, R. Goel, B. Kerbl, M. Steinberger, F. V. Carrasco, and F. De La Torre, "Taming 3dgs: High-quality radiance fields with limited resources," in *SIGGRAPH Asia 2024 Conference Papers*, 2024, pp. 1–11.

[33] S. Agarwal, K. Mierle, and T. C. S. Team, "Ceres Solver," 10 2023. [Online]. Available: https://github.com/ceres-solver/ceres-solver

[34] M. Burri, J. Nikolic, P. Gohl, T. Schneider, J. Rehder, S. Omari, M. W. Achtelik, and R. Siegwart, "The euroc micro aerial vehicle datasets," *The International Journal of Robotics Research*, vol. 35, no. 10, pp. 1157–1163, 2016.

[35] J. Sturm, N. Engelhard, F. Endres, W. Burgard, and D. Cremers, "A benchmark for the evaluation of rgb-d slam systems," in *2012 IEEE/RSJ international conference on intelligent robots and systems*. IEEE, 2012, pp. 573–580.

[36] T. Schops, T. Sattler, and M. Pollefeys, "Bad slam: Bundle adjusted direct rgb-d slam," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2019, pp. 134–144.

[37] A. Ndoye, A. Negre, N. Marchand, and F. Ruffier, "Replication Data for: VIGS-Fusion:Real-time Inertial Gaussian Splatting SLAM onboard the UAV," 2025. [Online]. Available: https://doi.org/10.57745/CI0K9G

[38] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," *arXiv preprint arXiv:1412.6980*, 2014.